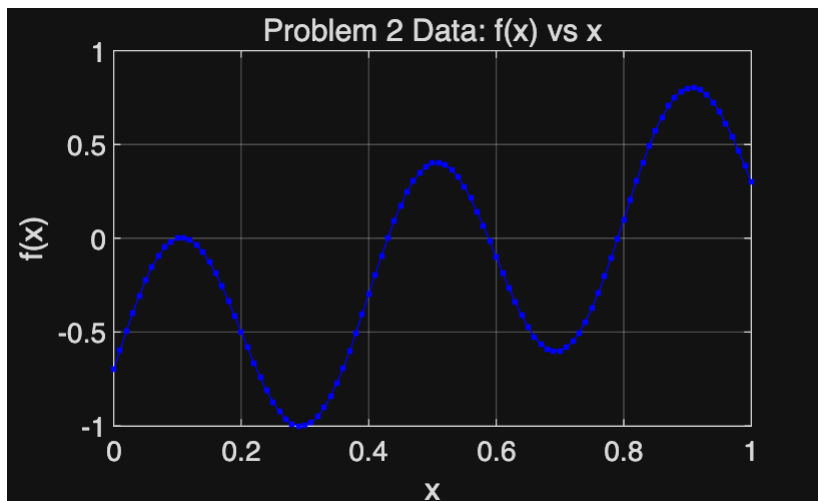


HW3 Question 2

```
clear; clc; close all;

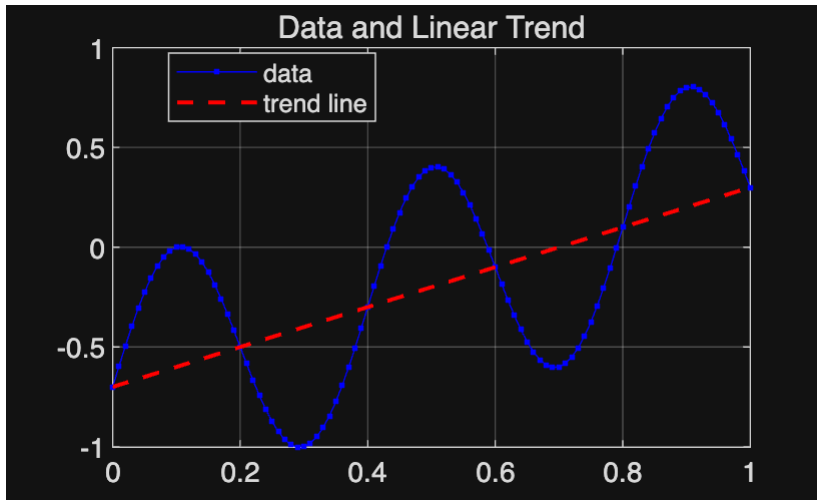
x = 0:0.01:1;
y = load("problem2.txt");
y = y(:);
N = numel(y);
assert(numel(x) == N, 'x and y must have the same length (101).');

figure(1);
plot(x, y, 'b.-');
xlabel('x'); ylabel('f(x)');
title('Problem 2 Data: f(x) vs x');
grid on;
```

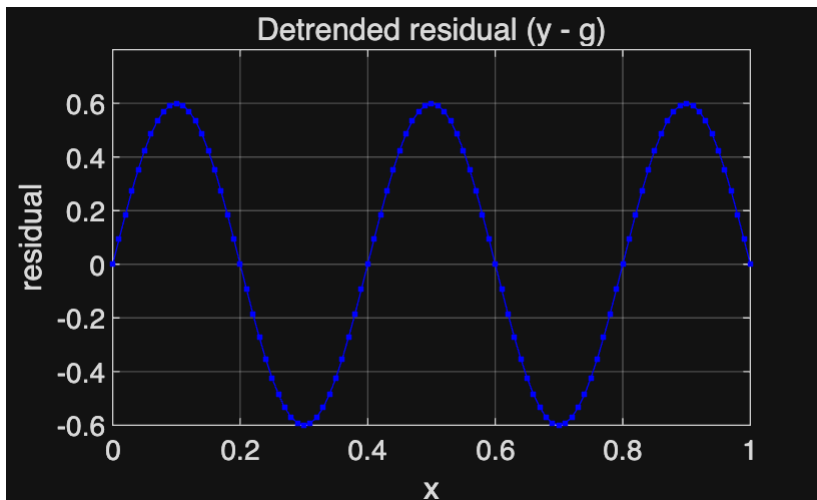


```
y0 = y(1);
y1 = y(end);
g = (y1 - y0) * x + y0;

figure(2);
plot(x, y, 'b.-'); hold on;
plot(x, g, 'r--', 'LineWidth', 1.5);
legend('data', 'trend line', 'Location', 'best');
title('Data and Linear Trend');
grid on;
```



```
t = y - g.';
figure(3);
plot(x, t, 'b.-');
xlabel('x'); ylabel('residual');
title('Detrended residual (y - g)');
grid on;
```



Now we need to approximate the curve. Since the curve has 2.5 period in the range, we could pick $k = 2.5$

```
k = 2.5;
Ak = [cos(2*pi*k*x(:)), sin(2*pi*k*x(:))];
beta = (Ak.'*Ak) \ (Ak.'*t);
bc = beta(1);
bs = beta(2);

yhat = g.' + Ak*beta;
```

```

a0 = y(1);
a1 = y(end) - y(1);
fprintf('k = %.3f\n', k);

```

```

k = 2.500

```

```

fprintf('a0 = %11.4e\n', a0);

```

```

a0 = -7.0000e-01

```

```

fprintf('a1 = %11.4e\n', a1);

```

```

a1 = 1.0000e+00

```

```

fprintf('bc = %11.4e\n', bc);

```

```

bc = -5.4530e-17

```

```

fprintf('bs = %11.4e\n', bs);

```

```

bs = 6.0000e-01

```

```

res = y - yhat;
mse = mean(res.^2);
R2 = 1 - sum(res.^2)/sum((y-mean(y)).^2);
fprintf('MSE = %.4e, R^2 = %.6f\n', mse, R2);

```

```

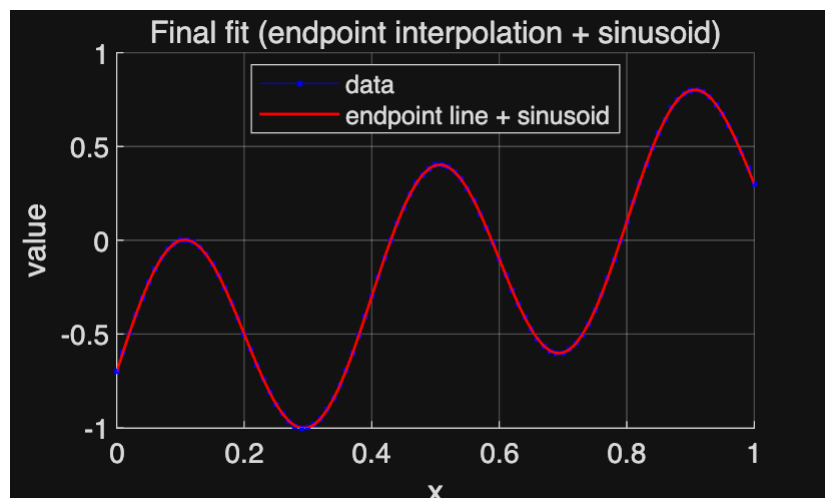
MSE = 3.3357e-31, R^2 = 1.000000

```

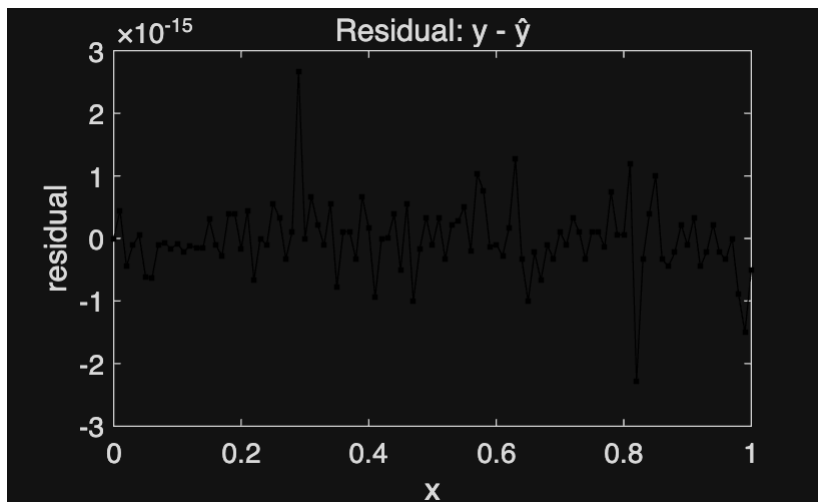
```

figure; hold on; grid on;
plot(x, y, 'b.-', 'DisplayName', 'data');
plot(x, yhat, 'r-', 'LineWidth', 1.0, 'DisplayName', 'endpoint line + sinusoid');
legend('Location', 'best');
xlabel('x'); ylabel('value');
title('Final fit (endpoint interpolation + sinusoid)');

```



```
figure; grid on;
plot(x, res, 'k.-');
xlabel('x'); ylabel('residual');
title('Residual: y -  $\hat{y}$ ');
```



```
fprintf('\nFinal fitted function:\n');
```

Final fitted function:

```
fprintf('f(x) = %.4e + (%.4e)*x + (%.4e)*cos(%.4f*pi*x) + (%.4e)*sin(%.4f*pi*x)\n',
...
      a0, a1, bc, 2*k, bs, 2*k);
```

```
f(x) = -7.0000e-01 + (1.0000e+00)*x + (-5.4530e-17)*cos(5.0000*pi*x) + (6.0000e-01)*sin(5.0000*pi*x)
```

Convert to amplitude–phase

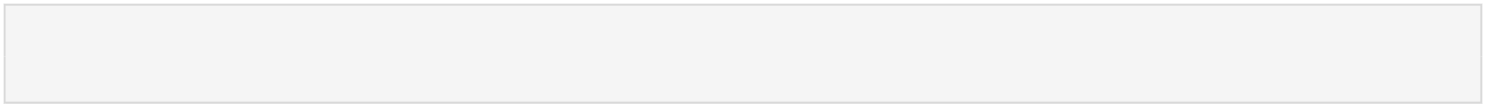
```
w = 2*pi*k;
A = hypot(bc, bs);
phi = atan2(bs, bc);
fprintf('Equivalent: f(x) = %.4e + (%.4e)*x + %.4e*cos(%.4f*x - %.4f)\n', ...
      a0, a1, A, w, phi);
```

```
Equivalent: f(x) = -7.0000e-01 + (1.0000e+00)*x + 6.0000e-01*cos(15.7080*x - 1.5708)
```

Error Metrics

```
res = y - yhat;
mse = mean(res.^2);
rmse = sqrt(mse);
R2 = 1 - sum(res.^2)/sum((y-mean(y)).^2);
fprintf('RMSE = %.4e, MSE = %.4e, R^2 = %.6f\n', rmse, mse, R2);
```

```
RMSE = 5.7755e-16, MSE = 3.3357e-31, R^2 = 1.000000
```



```

function hw3_q4()

close all; clc;

%% (a)
fprintf('==== (a) clear_table.txt: linear regression fit (c = -1) ====\\n');
data = load('clear_table.txt');
[a,b,d,meanDist] = fitPlaneLR_and_plot_xz(data, 'clear_table.txt (linear
regression)', true);
fprintf('Plane (a):  z = %.6f x + %.6f y + %.6f\\n', a, b, d);
fprintf('Mean orthogonal distance (m): %.6f\\n\\n', meanDist);

%% (b)
fprintf('==== (b) cluttered_table.txt: same linear regression fit (no
outlier handling) ====\\n');
data = load('cluttered_table.txt');
[a_b,b_b,d_b,meanDist_b] = fitPlaneLR_and_plot_xz(data,
'cluttered_table.txt (linear regression)', true);
fprintf('Plane (b):  z = %.6f x + %.6f y + %.6f\\n', a_b, b_b, d_b);
fprintf('Mean orthogonal distance (m): %.6f\\n', meanDist_b);
fprintf('Observation: Fit is skewed by clutter/outliers (e.g., cat). A
robust method is needed.\\n\\n');

%% (c)
fprintf('==== (c) cluttered_table.txt: robust plane via RANSAC ====\\n');
maxIters    = 2000;
inlierTol   = 0.01;
minInliers  = 0.5;
plotGrid    = true;

[plane_c, inlierMask_c, stats_c] = ransacPlane(data, maxIters, inlierTol,
minInliers);
plotPlaneOnCloud_xz(data, plane_c, inlierMask_c, 'cluttered_table.txt
(RANSAC dominant plane)', plotGrid);
fprintf('RANSAC plane (c):  ax + by - z + d = 0  ->  z = %.6f x + %.6f y +
%.6f\\n', plane_c(1), plane_c(2), plane_c(4));
fprintf('Inliers: %d / %d    |    Robust mean dist (inliers): %.6f m\\n\\n',
stats_c.numInliers, stats_c.N, stats_c.inlierMeanDist);

%% (d)
fprintf('==== (d) clean_hallway.txt: extract four planes via iterative
RANSAC ====\\n');
hall = load('clean_hallway.txt');
kPlanes = 4;
colors = lines(kPlanes);
figure('Name','clean_hallway: four planes (x-z base)');
hold on; grid on; axis equal; view(3);
title('clean_hallway: four dominant planes (x-z base, y vertical)',
'Interpreter','none');
xlabel('x [m]'); ylabel('z [m]'); zlabel('y [m]');

```

```

scatter3(hall(:,1), hall(:,3), hall(:,2), 4, [0.5 0.5 0.5], 'filled');

remainingMask = true(size(hall,1),1);
for k = 1:kPlanes
    [pl, inMask, st] = ransacPlane(hall(remainingMask,:), maxIters,
inlierTol, 0.1);
    idxRem = find(remainingMask);
    sel = idxRem(inMask);
    plotPlaneOnCloud_xz(hall, pl, inMask, sprintf('Plane %d',k), false,
colors(k,:));
    fprintf('Plane %d (d): z = %.6f x + %.6f y + %.6f | inliers = %d
meanDist = %.6f m\n', ...
        k, pl(1), pl(2), pl(4), st.numInliers, st.inlierMeanDist);
    remainingMask(sel) = false;
end
lock_axes_xzy(hall); drawnow;

%% (e)
fprintf('==== (e) cluttered_hallway.txt: four wall planes + smoothness &
safest surface ==== \n');
hall2 = load('cluttered_hallway.txt');
inlierTol_e = 0.008; kPlanes_e = 4;
residualMask = true(size(hall2,1),1);
colors = lines(kPlanes_e);
scores = zeros(kPlanes_e,1);
figure('Name','cluttered_hallway: four planes & smoothness (x-z base)');
hold on; grid on; axis equal; view(3);
title('cluttered_hallway: four planes with smoothness scores (x-z base)',
'Interpreter','none');
xlabel('x [m]'); ylabel('z [m]'); zlabel('y [m]');
scatter3(hall2(:,1), hall2(:,3), hall2(:,2), 4, [0.7 0.7 0.7], 'filled');

for k = 1:kPlanes_e
    [pl, inMask, st] = ransacPlane(hall2(residualMask,:), maxIters,
inlierTol_e, 0.05);
    idxRem = find(residualMask); sel = idxRem(inMask);
    dists = pointPlaneDistance(hall2(sel,:), pl);
    med = median(dists);
    scores(k) = sqrt(mean((dists - med).^2));
    plotPlaneOnCloud_xz(hall2, pl, inMask, sprintf('Plane %d (score=%.4g)',
k, scores(k)), false, colors(k,:));
    fprintf('Plane %d (e): z = %.6f x + %.6f y + %.6f | smoothness=%.6g\n',
k, pl(1), pl(2), pl(4), scores(k));
    residualMask(sel) = false;
end
[bestScore,bestIdx] = min(scores);
fprintf('\nSafest surface (smoothest) is Plane %d with score %.6g.\n',
bestIdx, bestScore);
title(sprintf('cluttered_hallway: safest = Plane %d, score=%.3g', bestIdx,
bestScore), 'Interpreter','none');

```

```

lock_axes_xzy(hall2); drawnow;

end

%% ----- Helper functions -----

function [a,b,d,meanDist] = fitPlaneLR_and_plot_xz(data, figTitle, doPlot)
x=data(:,1); y=data(:,2); z=data(:,3);
A=[x,y,ones(size(x))]; theta=A\z;
a=theta(1); b=theta(2); d=theta(3);
plane=[a,b,-1,d];
dist=pointPlaneDistance(data,plane);
meanDist=mean(dist);

if doPlot
    figure('Name',figTitle); hold on; grid on; axis equal; view(3);
    title([figTitle ' (x-z base, y vertical)'], 'Interpreter','none');
    xlabel('x [m]'); ylabel('z [m]'); zlabel('y [m]');
    scatter3(x,z,y,8,'b','filled');
    draw_plane_xz(plane,x,z,y,'FaceAlpha',0.35,'EdgeColor','none');
    lock_axes_xzy(data);
    txt=sprintf('z = %.3f x + %.3f y + %.3f\nMean dist = %.4f
m',a,b,d,meanDist);

    text(min(x),min(z),max(y),txt,'Interpreter','none','BackgroundColor','w');
end
end

function d=pointPlaneDistance(P,plane)
a=plane(1); b=plane(2); c=-1; d0=plane(4);
num=abs(a*P(:,1)+b*P(:,2)+c*P(:,3)+d0);
den=sqrt(a^2+b^2+c^2);
d=num/den;
end

function
[plane,inlierMask,stats]=ransacPlane(P,maxIters,inlierTol,minInlierFrac)
N=size(P,1); bestInliers=[]; bestPlane=[0 0 -1 0];
minInliersCount=max(3,ceil(minInlierFrac*N));
for it=1:maxIters
    idx=randperm(N,3); pts=P(idx,:);
    X=pts(:,1); Y=pts(:,2); Z=pts(:,3);
    M=[X,Y,ones(3,1)];
    if rank(M)<3, continue; end
    abdt=M\Z; a=abdt(1); b=abdt(2); d=abdt(3);
    pl=[a,b,-1,d];
    dists=pointPlaneDistance(P,pl);
    inliers=find(dists<=inlierTol);
    if numel(inliers)>numel(bestInliers)
        bestInliers=inliers(:); bestPlane=pl;
    end
end
end

```



```

    end
end
if isempty(bestInliers)
    theta=[P(:,1),P(:,2),ones(N,1)]\P(:,3);
    plane=[theta(1),theta(2),-1,theta(3)];
    inlierMask=true(N,1);
else
    Xi=P(bestInliers,1); Yi=P(bestInliers,2); Zi=P(bestInliers,3);
    theta=[Xi,Yi,ones(numel(bestInliers),1)]\Zi;
    plane=[theta(1),theta(2),-1,theta(3)];
    dAll=pointPlaneDistance(P,plane);
    inlierMask=(dAll<=inlierTol);
end
dIn=pointPlaneDistance(P(inlierMask,:),plane);
stats.numInliers=sum(inlierMask); stats.N=N; stats.inlierMeanDist=mean(dIn);
end

function
plotPlaneOnCloud_xz(P,plane,inlierMask,figTitle,makeNewFig,faceColor)
if nargin<5,makeNewFig=true;end
if nargin<6,faceColor=[1 0 0];end
x=P(:,1); y=P(:,2); z=P(:,3);
if makeNewFig
    figure('Name',figTitle); hold on; grid on; axis equal; view(3);
    title([figTitle ' (x-z base, y vertical)'], 'Interpreter','none');
    xlabel('x [m]'); ylabel('z [m]'); zlabel('y [m]');
end
scatter3(x(~inlierMask),z(~inlierMask),y(~inlierMask),4,[0.7 0.7
0.7], 'filled');
scatter3(x(inlierMask),z(inlierMask),y(inlierMask),6,[0 0.4 1], 'filled');
draw_plane_xz(plane,x,z,y,'FaceAlpha',0.25,'EdgeColor','none','FaceColor',faceColor);
lock_axes_xzy(P);
end

function draw_plane_xz(plane,x,z,y,varargin)
a=plane(1); b=plane(2); c=-1; d=plane(4);
if abs(b)>1e-6
    xr=linspace(min(x),max(x),30);
    zr=linspace(min(z),max(z),30);
    [X,Z]=meshgrid(xr,zr);
    Y=(Z - a*X - d)/b;
    lo=prctile(y,2); hi=prctile(y,98);
    Y=max(min(Y,hi),lo);
    surf(X,Z,Y,varargin{:});
else
    xr=linspace(min(x),max(x),30);
    yr=linspace(min(y),max(y),30);
    [X,Y]=meshgrid(xr,yr);
    Z=a*X + b*Y + d;

```

```
    surf(X,Z,Y,varargin{:});  
end  
end  
  
function lock_axes_xzy(P)  
x=P(:,1); y=P(:,2); z=P(:,3);  
xlim([min(x) max(x)]); ylim([min(z) max(z)]); zlim([min(y) max(y)]);  
pbaspect([1 1 1]);  
end
```

==== (a) clear_table.txt: linear regression fit (c = -1) ====

Plane (a): $z = -1.793057 x + -18.745235 y + 9.439102$

Mean orthogonal distance (m): 0.002855

==== (b) cluttered_table.txt: same linear regression fit (no outlier handling) ====

Plane (b): $z = -0.200116 x + -2.022388 y + 2.656737$

Mean orthogonal distance (m): 0.077150

Observation: Fit is skewed by clutter/outliers (e.g., cat). A robust method is needed.

==== (c) cluttered_table.txt: robust plane via RANSAC ====

RANSAC plane (c): $ax + by - z + d = 0 \rightarrow z = -1.897461 x + -18.749247 y + 9.451068$

Inliers: 3482 / 3888 | Robust mean dist (inliers): 0.002729 m

==== (d) clean_hallway.txt: extract four planes via iterative RANSAC ====

Plane 1 (d): $z = 4.120215 x + -23.935959 y + 44.153596$ | inliers = 4231 meanDist = 0.003745 m

Plane 2 (d): $z = 4.160018 x + -24.248826 y + 5.305229$ | inliers = 4187 meanDist = 0.003264 m

Plane 3 (d): $z = 4.702763 x + 0.851919 y + -1.029738$ | inliers = 3974 meanDist = 0.004896 m

Plane 4 (d): $z = 4.610084 x + 0.808254 y + -8.614342$ | inliers = 2432 meanDist = 0.005025 m

==== (e) cluttered_hallway.txt: four wall planes + smoothness & safest surface ====

Plane 1 (e): $z = 0.126985 x + 9.925235 y + -5.838017$ | smoothness=0.0021248

Plane 2 (e): $z = 0.209725 x + 10.873949 y + 5.794176$ | smoothness=0.00215384

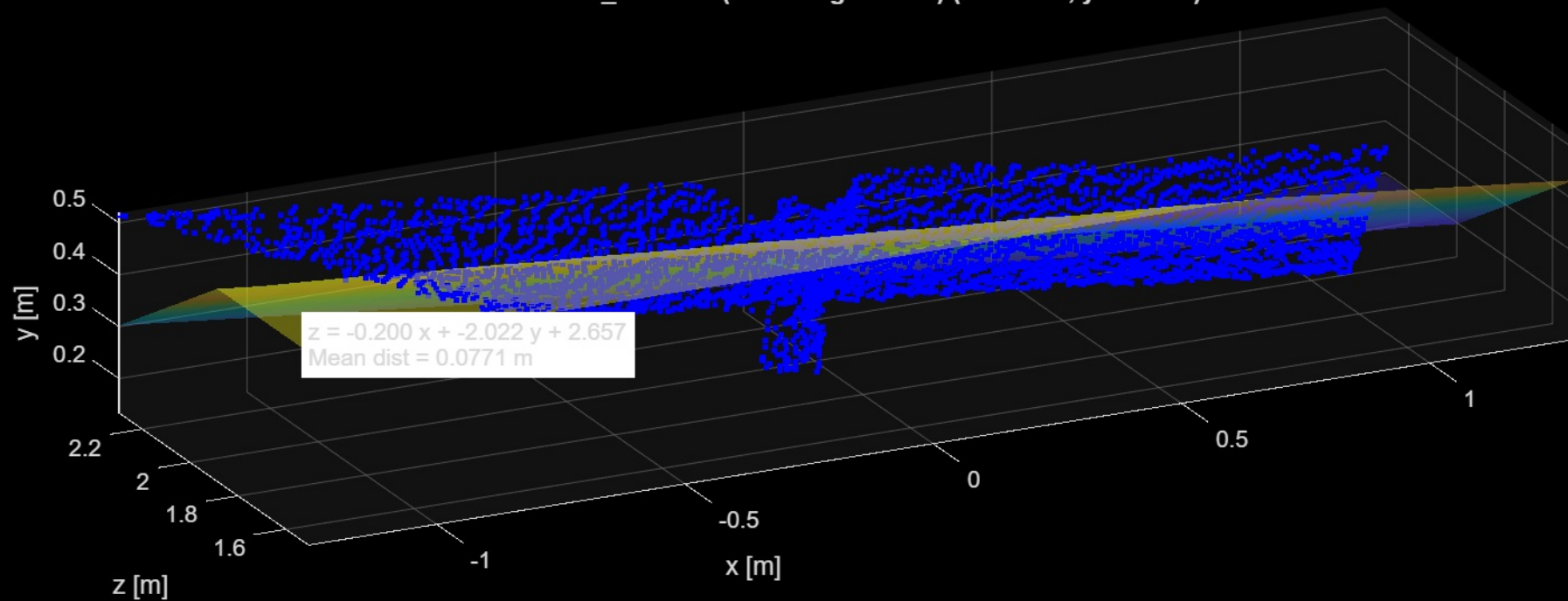
Plane 3 (e): $z = 10.649688 x + -0.394236 y + 18.801792$ | smoothness=0.00230379

Plane 4 (e): $z = 18.904757 x + -0.146488 y + -19.569269$ | smoothness=0.00219128

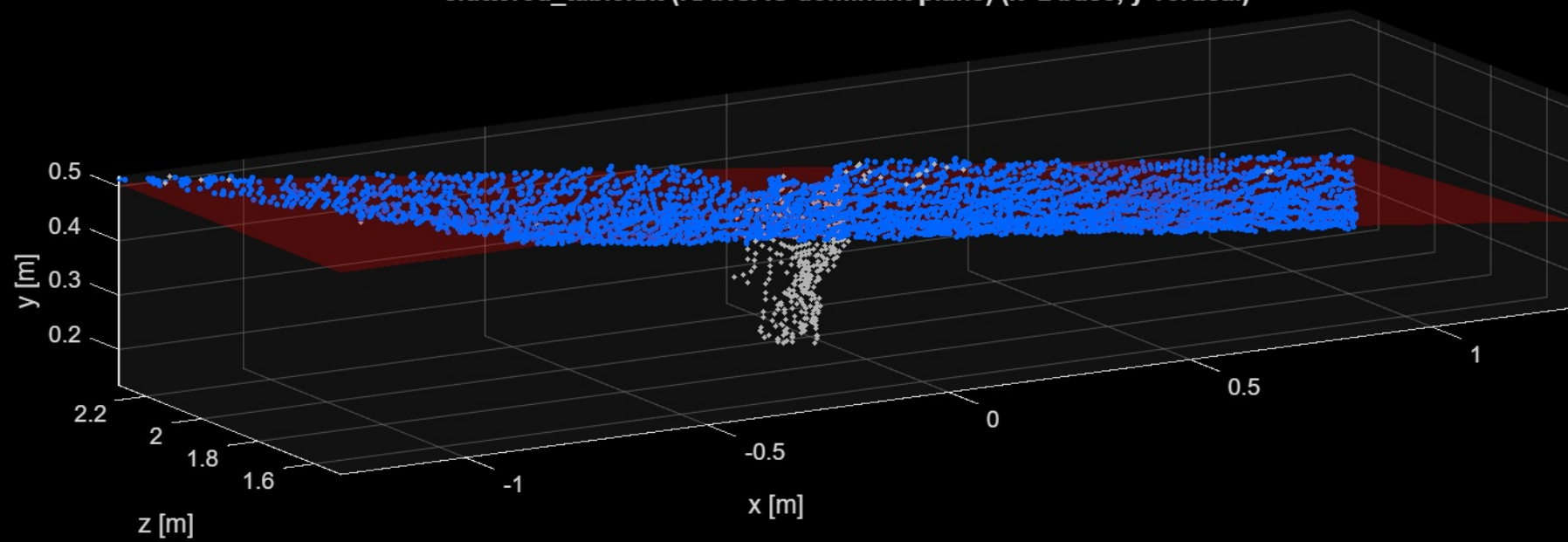
Safest surface (smoothest) is Plane 1 with score 0.0021248.

>>

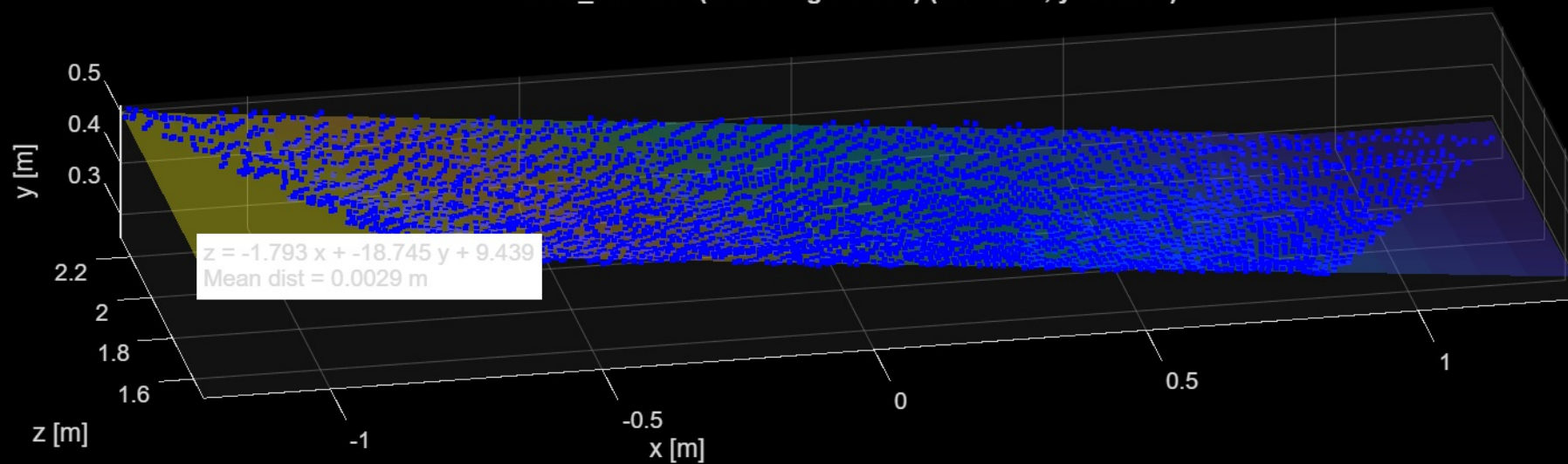
cluttered_table.txt (linear regression) (x-z base, y vertical)



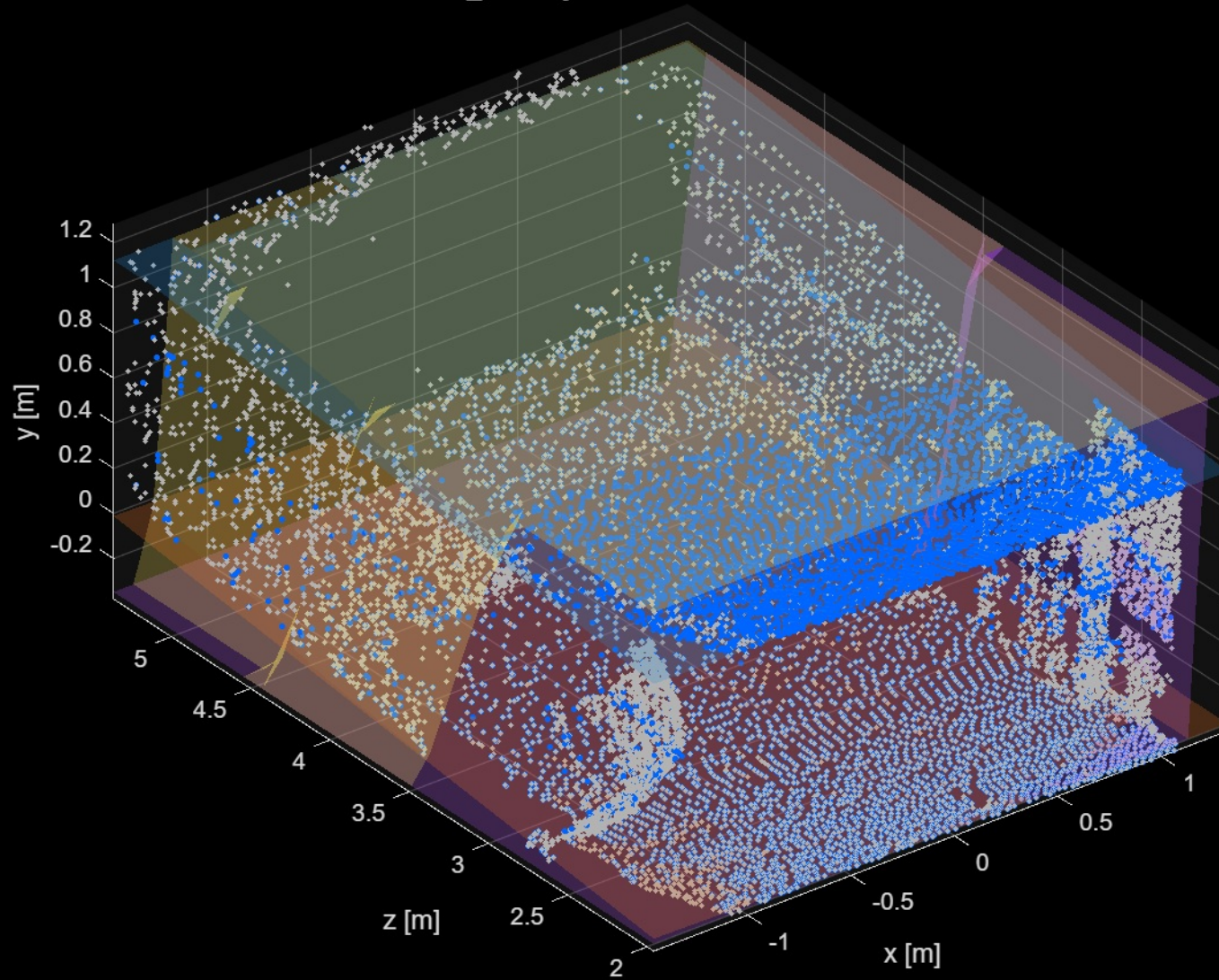
cluttered_table.txt (RANSAC dominant plane) (x-z base, y vertical)



clear_table.txt (linear regression) (x-z base, y vertical)



cluttered_hallway: safest = Plane 1, score=0.00212



clean_hallway: four dominant planes (x-z base, y vertical)

